

Our 1st logical model: Boolean logic

Aka classical propositional logic

Objects to be modelled: **declarative sentences (propositions)**.



"The evidence obtained is admissible."

"All Martians like pepperoni on their pizza."

"Every > 2 even number is the sum of two primes."

"He only relaxes on Sundays."

NOT declarative: *"Hi, how are you?"*

"Don't do that!"

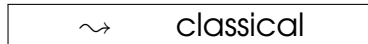
"May the Force be with you."

Modelling with classical propositional logic

The chosen essential feature of declarative sentences is as follows:

In each situation, they are either **true** or **false**.

- There are no third or further cases.
- A declarative sentence cannot be **true** AND **false** at the same time.



There are many other *propositional* logics:

- intuitionistic
- paraconsistent
- many-valued
- ...

Boolean logic: alphabet

The alphabet of Boolean logic consists of the following symbols:

- a fixed infinite set of propositional variables (we typically write p, q, r, x, y, z or $p_1, p_2, \dots$)
- the logical connectives
 - $\neg$ (*negation*, reads as 'not')
 - $\wedge$ (*conjunction*, reads as 'and')
 - $\vee$ (*disjunction*, reads as 'or')
 - $\rightarrow$ (*implication*, reads as 'implies' or 'if... then')
 - $\leftrightarrow$ (*bi-implication*, reads as 'if and only if' or 'iff')
- the auxiliary symbols $)$ and $($.

Syntax of Boolean logic: formulas

Well-formed **formulas** of Boolean logic are built up inductively as follows:

- *Base case:* Any propositional variable is a formula.
- *Inductive step:* If A and B are formulas then so are

$$(\neg A) \quad (A \wedge B) \quad (A \vee B) \quad (A \rightarrow B) \quad (A \leftrightarrow B)$$

- *Closure:* Nothing else is a formula.

Examples:

$$((p \wedge q) \rightarrow ((\neg r) \vee q))$$

$$(((\neg p) \wedge (\neg(\neg q))) \rightarrow r)$$

Notational conventions

- We omit the outermost parentheses of formulas.
- $\neg$ binds more tightly than $\vee$, $\wedge$, $\rightarrow$, and $\leftrightarrow$, so we omit the parentheses in occurrences of $(\neg A)$.

Examples:

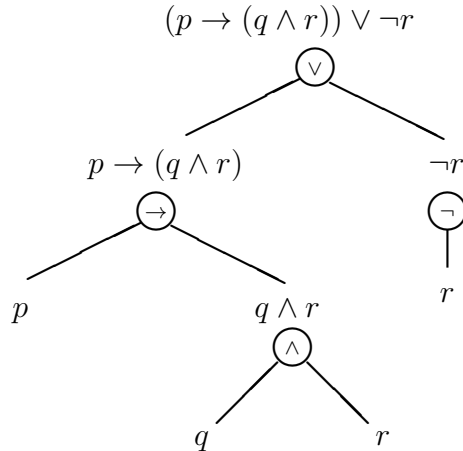
we write $(p \wedge q) \rightarrow (\neg r \vee q)$ for $((p \wedge q) \rightarrow ((\neg r) \vee q))$

we write $(\neg p \wedge \neg \neg q) \rightarrow r$ for $((\neg p) \wedge (\neg(\neg q))) \rightarrow r$

But always use brackets if in any doubt!

Parse tree of a formula

Aka **syntactic decomposition tree**:



Any well-formed formula can be decomposed this way.

The formulas occurring as nodes of the parse tree are called **subformulas** of the initial formula.

Semantics of Boolean logic: interpretations

Interpretations are intended to 'model' the possible 'circumstances' in which a formal expression (a formula) can be evaluated.

An interpretation is a function assigning to each propositional variable a truth-value '1' (true) or '0' (false).

Now we can connect syntax and semantics ('model checking'):

Given an interpretation and a formula, we can compute the truth-value of the formula in a 'uniform' way. It will *only* depend on two things:

- (1) the truth-values of the propositional variables of the formula in the given interpretation, and
- (2) the truth-tables of the logical connectives in the formula (that give an unambiguous 'answer' for any interpretation).

Boolean logic: truth-tables

A	$\neg A$
1	0
0	1

A	B	$A \wedge B$	$A \vee B$	$A \rightarrow B$	$A \leftrightarrow B$
1	1	1	1	1	1
1	0	0	1	0	0
0	1	0	1	1	0
0	0	0	0	1	1

Notes about the truth-tables of the connectives

The truth-tables of the logical connectives $\vee$, $\wedge$ and $\rightarrow$ of Boolean logic do not always 'mimic' the corresponding natural language wordings.

- $A \vee B$ is NOT **exclusive** 'or': it can be **1** even if BOTH A and B are **1**.

"I'll go either to the west or to the east."

- $A \wedge B$ and $B \wedge A$ mean the same.

"Bob became sick and he visited the doctor."

"Bob visited the doctor and he became sick."

- 'Strange' formulas of the form $A \rightarrow B$ can be **1**.

"If the Moon is made of green cheese then $2 \times 2 = 4$."

"If the Moon is made of green cheese then $2 \times 2 = 5$."

"If London is in the U.K. then $2 \times 2 = 4$."

There are other logics modelling these features more faithfully.

Exercise 1.1: Model checking

Compute the truth-value of the formula

$$(p_2 \rightarrow (p_1 \vee p_6)) \rightarrow ((p_2 \wedge p_1) \rightarrow \neg p_6)$$

in each of the following interpretations:

(a) $p_1 \mapsto \mathbf{0}, \quad p_2 \mapsto \mathbf{1}, \quad p_6 \mapsto \mathbf{0}$

(b) $p_1 \mapsto \mathbf{1}, \quad p_2 \mapsto \mathbf{1}, \quad p_6 \mapsto \mathbf{0}$

(c) $p_1 \mapsto \mathbf{0}, \quad p_2 \mapsto \mathbf{0}, \quad p_6 \mapsto \mathbf{0}$

(d) $p_1 \mapsto \mathbf{1}, \quad p_2 \mapsto \mathbf{1}, \quad p_6 \mapsto \mathbf{1}$

Boolean logic: Validity, satisfiability and consistency

- A formula is called **valid** if it evaluates to **1** in every interpretation. (Valid formulas of Boolean logic are also known as **tautologies**.)
- A formula is **satisfiable** if *there exists* an interpretation that makes it **1**.

if A is valid then A is satisfiable

Validity and satisfiability can be reduced to each other:

A is valid iff $\neg A$ is not satisfiable

A is satisfiable iff $\neg A$ is not valid

- A set S of formulas is **consistent** if *there exists* an interpretation that evaluates every formula in S to **1**. (It must be the **same** interpretation **for all** formulas in S .)

Boolean logic: Logical consequence

We say that a formula B is a logical consequence of formulas $A_1, \dots, A_n$ (or, B logically follows from $A_1, \dots, A_n$), if in every interpretation where each and every of $A_1, \dots, A_n$ evaluates to **1**, B also evaluates to **1**.

Then we also say that the 'argument of deducing the **conclusion** B from the **assumptions** $A_1, \dots, A_n$ ' is logically correct.

Notation:

$$A_1, \dots, A_n \models B$$

Examples: $p, q \vee \neg p \models q$

$$(p \wedge q) \rightarrow r \models (p \rightarrow r) \vee (q \rightarrow r)$$

- Logical consequence can be reduced to validity (*Deduction Theorem*):

$$A_1, \dots, A_n \models B \quad \text{iff} \quad (A_1 \wedge \dots \wedge A_n) \rightarrow B \text{ is a valid formula.}$$

- Logical consequence can be reduced to consistency:

$$A_1, \dots, A_n \models B \quad \text{iff} \quad \text{the set } \{A_1, \dots, A_n, \neg B\} \text{ of formulas is not consistent.}$$

Boolean logic: Logical equivalence

We say that formulas A and B are logically equivalent, if in every interpretation they evaluate to the *same* truth-value. That is, in every interpretation

- either both A and B evaluate to **1**,
- or both A and B evaluate to **0**.

Notation:

$$A \equiv B$$

Well-known examples:

$$A \wedge (B \vee C) \equiv (A \wedge B) \vee (A \wedge C)$$

$$A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C)$$

$$A \equiv \neg\neg A$$

$$A \rightarrow B \equiv \neg A \vee B$$

$$\neg(A \wedge B) \equiv \neg A \vee \neg B$$

$$\neg(A \vee B) \equiv \neg A \wedge \neg B$$

$$A \rightarrow B \equiv \neg B \rightarrow \neg A$$

- Logical equivalence can also be reduced to validity:

$A \equiv B$ iff $A \leftrightarrow B$ is a valid formula.

Introducing 1 and 0 as formulas

It is often convenient to consider **1** and **0** not only as truth-values, but also as **formulas**:

- You can consider **1** as an **abbreviation** for your favourite valid formula, such as $p \rightarrow p$ or $q \vee \neg q$.
- Then, you can consider **0** as an **abbreviation** for $\neg(p \rightarrow p)$ or $\neg(q \vee \neg q)$.

This way we can have some more examples of logical equivalences:

$$A \vee \mathbf{1} \equiv \mathbf{1}$$

$$A \wedge \mathbf{1} \equiv A$$

$$A \vee \mathbf{0} \equiv A$$

$$A \wedge \mathbf{0} \equiv \mathbf{0}$$

$$\mathbf{1} \rightarrow A \equiv A$$

$$A \rightarrow \mathbf{1} \equiv \mathbf{1}$$

$$\mathbf{0} \rightarrow A \equiv \mathbf{1}$$

$$A \rightarrow \mathbf{0} \equiv \neg A \quad \dots$$

Validity and effectiveness

Is there an effective procedure (an algorithm) that decides whether a given formula is valid? By an *algorithm* we mean something meeting the following conditions:

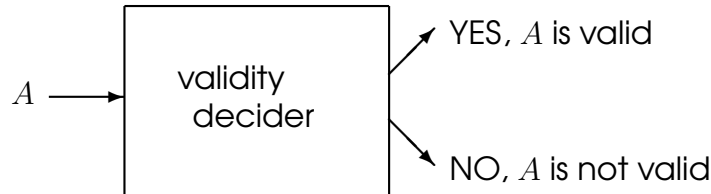
- It must consist of exact step-by-step instructions.
- These instructions must not demand any brilliance or originality on the part of the person (or machine) following them. A diligent clerk (who does not like to think but is very good in following directions) or a computer (that does not think at all) should be able to execute the instructions.
- The procedure should be '**universal**': given **any** formula A , after a finite number of steps the procedure should produce either a "YES, A is valid" or "NO, A is not valid" answer.

In other words, we need a computer program that can take **any** formula as input, and outputs YES if the input formula is valid, and NO if it is not valid.

Boolean logic is decidable

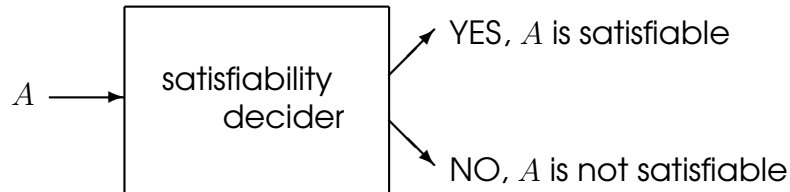
The **validity problem** for Boolean logic is **decidable**:

It is possible to write a computer program which takes ANY formula as input, and decides whether or not the formula is valid.



The **satisfiability problem** for Boolean logic is also **decidable**:

It is possible to write a computer program which takes ANY formula as input, and decides whether or not the formula is satisfiable.



Boolean logic is decidable: why?

Actually, there are many different algorithms known.

Here is our first one, the truth-table algorithm:

In every formula there are finitely many propositional variables, so there are finitely many interpretations 'relevant' to the formula. We just take each and every of them one by one, and compute the truth-value of the input formula in each of them. Then:

- Deciding validity:

If our formula evaluates to **1** in every interpretation, then we say YES.
Otherwise, we say NO.

- Deciding satisfiability:

If we find *at least one* interpretation where our formula evaluates to **1**, then we can stop and say YES. Otherwise, we have to continue until we have exhausted all possible interpretations. If all of them evaluate our formula to **0**, then we say NO at the end.

Truth-table algorithm: an example

Is the following formula valid? Is it satisfiable?

$$\boxed{((p \vee q) \rightarrow r) \rightarrow ((\neg p \rightarrow r) \wedge (q \rightarrow r))}$$

Let's compute the formula's *truth-table*:

p	q	r	$((p \vee q) \rightarrow r) \rightarrow ((\neg p \rightarrow r) \wedge (q \rightarrow r))$
1	1	1	1
1	1	0	1
1	0	1	1
1	0	0	1
0	1	1	1
0	1	0	1
0	0	1	1
0	0	0	0

So this formula is **not valid**, but **satisfiable**.

Truth-table algorithm: time complexity

Ok, we can always do it, but how long does it take?

The number of different interpretations is exponential in the number of propositional variables occurring in the formula:

n propositional variables $\leadsto 2^n$ lines in the truth-table

In the worst case, if we are unlucky, we need to check all possible interpretations to give an answer. Suppose we can generate one line in the table super-fast: million lines per sec. Then, say, if $n = 80$ (not a large number in a real life database), then we would need 'merely' 2^{80} microseconds to form the whole truth-table. How long is that?

2^{80} microseconds is about **38 billion years** $>$ age of the universe

BAD news.

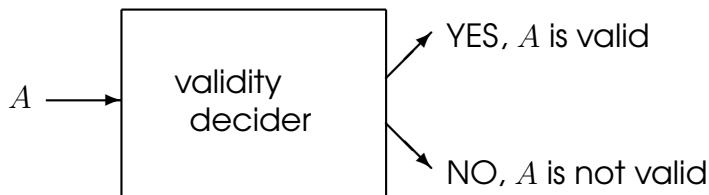
Is there something faster?

So far nobody could come up with a method that is faster in the worst case. However:

- There are many different algorithms for deciding validity and satisfiability that work fast in many cases. Each algorithm uses different ways of representing formulas, and different optimisation strategies. But in each case one can find 'bad examples:' formulas on which the particular algorithm must perform exponentially many steps.
- The satisfiability/validity problems of Boolean logic are connected to many other, very different problems. It is known that if somebody could come up with a faster, *non-exponential* (that is, *polynomial*) solution to **any** of these, this would give quick solutions to **all** these other problems. (E.g., such a quick solution would break the RSA cryptosystem.)

This general question is called the '**P versus NP problem**,' the most prominent open problem in theoretical computer science today.

Another validity-deciding algorithm: the tableau procedure



Given any formula A , it is either valid or not valid. If A is not valid, then there is an interpretation where A evaluates to **0**. We call such an interpretation a **counter-interpretation for A** .

Given any formula A , the **tableau procedure** tries to construct such a counter-interpretation for A via a finite number of simple but exactly given steps. These steps will not require any 'thinking', so even a computer can be programmed to follow them.

The tableau procedure as a validity deciding algorithm

The tableau procedure will be such that:

- It is **sound**:

If the procedure claims that something is a counter-interpretation for the input-formula, then it indeed is. So when the procedure says NO, then the formula is indeed *not valid*.

- It is **complete**:

The procedure always finds a counter-interpretation if it exists. So if the procedure does not succeed, then there is no counter-interpretation for the input-formula, that is, it is valid. This is when the procedure says YES.

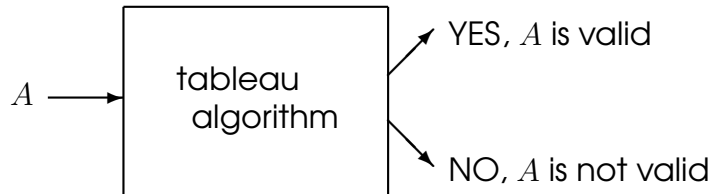


Tableau procedure: how to start

Question: Is the formula $((p \rightarrow q) \rightarrow p) \rightarrow p$ valid?

We form a two-column **tableau**, where we list

- the formulas that we want to be **1** in the **left** column, and
- the formulas that we want to be **0** in the **right** column.

At the first step, as we are looking for a counter-interpretation for our formula, all we want is $((p \rightarrow q) \rightarrow p) \rightarrow p$ to be **0**. So the initial tableau looks like this:

	$((p \rightarrow q) \rightarrow p) \rightarrow p$
--	---

Tableau procedure: sketch

Starting from the initial tableau, at each subsequent step we ‘unfold’ some formula in the tableau to simpler subformulas, following some simple rules. By ‘unfolding’ the formulas, we obtain simpler and simpler formulas, and sooner or later the procedure stops.

There can be **two** reasons for the procedure to stop:

- (1) There are no ‘unfoldable’ formulas left, we have ‘reached’ the propositional variables. In this case we can turn the tableau to a counter-interpretation for the initial formula.

This is when the procedure says: NO, the initial formula is **not valid**.

- (2) We obtain a **contradictory tableau**: a tableau where the very same formula occurs in **both** its left (true) and right (false) column. This will mean that there is no counter-interpretation for the initial formula.

This is when the procedure says: YES, the initial formula is **valid**.

While the truth-table algorithm works *inside-out*, the tableau algorithm works *outside-in*.

Tableau procedure: the subsequent steps

Starting from the initial tableau, at each subsequent step:

- We choose **ANY** formula in one of the columns
 - that is not a propositional variable, and
 - that hasn't been chosen before (until there is such).
- We find its **outermost** logical connective (the last connective used when building the formula from variables).
- Then apply the corresponding **rule** that puts some subformulas into the columns of the tableau.

There are altogether 10 rules:

2 rules for each of the 5 logical connectives $\neg$, $\wedge$, $\vee$, $\rightarrow$, and $\leftrightarrow$.

- Say, we apply the $\wedge_{false}$ rule, if we choose a formula in the right (false) column and $\wedge$ is its outermost connective.
- Or, we apply the $\rightarrow_{true}$ rule, if we choose a formula in the left (true) column and $\rightarrow$ is its outermost connective.

Tableau procedure: the $\rightarrow_{false}$ rule

We continue the tableau on slide 41.

We have only one formula in our tableau, so let's choose it. The outermost connective of the formula $((p \rightarrow q) \rightarrow p) \rightarrow p$ is $\rightarrow$. So the $\rightarrow_{false}$ rule applies:

A	$A \rightarrow B$ B
-----	--------------------------

(In order to memorise the $\rightarrow_{false}$ rule, we should consult the truth-table of $\rightarrow$:

A formula of the form $A \rightarrow B$ can be **0** only if A is **1** and B is **0**.)

The $\rightarrow_{false}$ rule applied to the formula $((p \rightarrow q) \rightarrow p) \rightarrow p$ results in the extended tableau:

$(p \rightarrow q) \rightarrow p$	$((p \rightarrow q) \rightarrow p) \rightarrow p$ p
-----------------------------------	--

Tableau procedure: the $\rightarrow_{true}$ rule

We have three formulas. The formulas in the right column are either have been chosen or variables. So let's choose the formula in the left column:

The outermost connective of the formula $(p \rightarrow q) \rightarrow p$ is $\rightarrow$.

So the $\rightarrow_{true}$ rule applies. According to this rule, there are two cases:

either	<table><tr><td>$A \rightarrow B^\bullet$</td><td>A</td></tr></table>	$A \rightarrow B^\bullet$	A	or	<table><tr><td>$A \rightarrow B^\bullet$</td><td>B</td></tr></table>	$A \rightarrow B^\bullet$	B
$A \rightarrow B^\bullet$	A						
$A \rightarrow B^\bullet$	B						

Applying the $\rightarrow_{true}$ rule to $(p \rightarrow q) \rightarrow p$: We mark the chosen formula with a $\bullet$, in order to indicate that there are different cases to pursue.

Case 1:

$(p \rightarrow q) \rightarrow p^\bullet$	$((p \rightarrow q) \rightarrow p) \rightarrow p$ p $p \rightarrow q$
---	---

Case 2:

$(p \rightarrow q) \rightarrow p^\bullet$ p	$((p \rightarrow q) \rightarrow p) \rightarrow p$ p
--	--

Tableau procedure: the example continued

Now we have to pursue the two cases independently, in parallel.

Case 1:

$(p \rightarrow q) \rightarrow p^\bullet$	$((p \rightarrow q) \rightarrow p) \rightarrow p$
	p
	$p \rightarrow q$

We apply the $\rightarrow_{false}$ rule again, now to $p \rightarrow q$:

$(p \rightarrow q) \rightarrow p^\bullet$	$((p \rightarrow q) \rightarrow p) \rightarrow p$
	p
	$p \rightarrow q$
p	q

This is contradictory: we have p in both columns, meaning it should be both **1** and **0** at the same time. Though this case is closed unsuccessfully, we cannot conclude that there is no counter-interpretation, since we still have another case.

Case 2:

$(p \rightarrow q) \rightarrow p^\bullet$	$((p \rightarrow q) \rightarrow p) \rightarrow p$
p	p

Again contradictory: p in both columns. There are no more cases, meaning we failed to find a counter-interpretation. So $((p \rightarrow q) \rightarrow p) \rightarrow p$ is valid.

Tableau procedure: summary of the rules I

• $\neg_{true}$ rule:

$\neg A$	A
----------	-----

• $\neg_{false}$ rule:

A	$\neg A$
-----	----------

• $\wedge_{true}$ rule:

$A \wedge B$	
A	
B	

• $\vee_{false}$ rule:

	$A \vee B$
	A
	B

• $\wedge_{false}$ rule: There are two cases:

either

	$A \wedge B^\bullet$
	A

or

	$A \wedge B^\bullet$
	B

• $\vee_{true}$ rule: There are two cases:

either

$A \vee B^\bullet$	
A	

or

$A \vee B^\bullet$	
B	

Tableau procedure: summary of the rules II

- $\rightarrow_{true}$ rule: There are two cases:

either

$A \rightarrow B^\bullet$	A
---------------------------	-----

or

$A \rightarrow B^\bullet$ B	
----------------------------------	--

- $\rightarrow_{false}$ rule:

A	$A \rightarrow B$ B
-----	--------------------------

- $\leftrightarrow_{true}$ rule: There are two cases:

either

$A \leftrightarrow B^\bullet$ A B	
---	--

or

$A \leftrightarrow B^\bullet$	A B
-------------------------------	------------

- $\leftrightarrow_{false}$ rule: There are two cases:

either

A	$A \leftrightarrow B^\bullet$ B
-----	--------------------------------------

or

B	$A \leftrightarrow B^\bullet$ A
-----	--------------------------------------

Tableau procedure: another example

Question: Is the formula $(r \wedge (\neg p \vee \neg q)) \rightarrow ((p \vee \neg q) \wedge r)$ valid?

We apply the $\rightarrow_{false}$ rule, and then the $\wedge_{true}$ rule to $r \wedge (\neg p \vee \neg q)$:

$r \wedge (\neg p \vee \neg q)$ r $\neg p \vee \neg q$	$(r \wedge (\neg p \vee \neg q)) \rightarrow ((p \vee \neg q) \wedge r)$ $(p \vee \neg q) \wedge r$
--	--

Then the $\wedge_{false}$ rule applied to $(p \vee \neg q) \wedge r$ results in two cases:

Case 1:

$r \wedge (\neg p \vee \neg q)$ r $\neg p \vee \neg q$	$(r \wedge (\neg p \vee \neg q)) \rightarrow ((p \vee \neg q) \wedge r)$ $(p \vee \neg q) \wedge r^\bullet$ $p \vee \neg q$
--	---

Case 2:

$r \wedge (\neg p \vee \neg q)$ r $\neg p \vee \neg q$	$(r \wedge (\neg p \vee \neg q)) \rightarrow ((p \vee \neg q) \wedge r)$ $(p \vee \neg q) \wedge r^\bullet$ r
--	---

Tableau procedure: another example (cont.)

Now we have to pursue Case 1 and Case 2 independently, in parallel.

Case 1: We apply the $\vee_{false}$ rule to $p \vee \neg q$:

$r \wedge (\neg p \vee \neg q)$	$(r \wedge (\neg p \vee \neg q)) \rightarrow ((p \vee \neg q) \wedge r)$
r	$(p \vee \neg q) \wedge r$
$\neg p \vee \neg q$	$p \vee \neg q$
	p
	$\neg q$

Then, the application of the $\vee_{true}$ rule to $\neg p \vee \neg q$ results in two subcases:

Case 1.1:

$r \wedge (\neg p \vee \neg q)$	$(r \wedge (\neg p \vee \neg q)) \rightarrow ((p \vee \neg q) \wedge r)$
r	$(p \vee \neg q) \wedge r$
$\neg p \vee \neg q^\bullet$	$p \vee \neg q$
$\neg p$	p
	$\neg q$

Case 1.2:

$r \wedge (\neg p \vee \neg q)$	$(r \wedge (\neg p \vee \neg q)) \rightarrow ((p \vee \neg q) \wedge r)$
r	$(p \vee \neg q) \wedge r$
$\neg p \vee \neg q^\bullet$	$p \vee \neg q$
$\neg q$	p
	$\neg q$

Tableau procedure: another example (cont.)

Case 1.1: (cont.) We apply the $\neg_{true}$ and the $\neg_{false}$ rules:

$r \wedge (\neg p \vee \neg q)$	$(r \wedge (\neg p \vee \neg q)) \rightarrow ((p \vee \neg q) \wedge r)$
r	$(p \vee \neg q) \wedge r$
$\neg p \vee \neg q$	$p \vee \neg q$
$\neg p$	p
q	$\neg q$
	p

There are no more formulas to unfold. This case gives us a counter-interpretation: We simply evaluate those propositional variables that are in the left column to **1**, and those that are in the right column to **0**:

$$p \mapsto \mathbf{0} \qquad q \mapsto \mathbf{1} \qquad r \mapsto \mathbf{1}$$

This time we don't have to bother with the other open cases (Case 1.2 and Case 2), as we have already managed to find a counter-interpretation, showing that $(r \wedge (\neg p \vee \neg q)) \rightarrow ((p \vee \neg q) \wedge r)$ is not valid.

Summary: how to use tableaux for deciding validity?

Starting from the initial tableau, at each subsequent step we 'unfold' some formula in the tableau to simpler subformulas, following some simple rules. Some of the rules result in different cases.

When to stop, do we always have to pursue all cases?

There can be **two** reasons for the procedure to stop:

- (1) If we find **at least one case** where there are no 'unfoldable' formulas left, because we have 'reached' the propositional variables. In this case our tableau can be turned to a counter-interpretation for the initial formula. This is when the procedure says: NO, the initial formula is **not valid**. We do not have to pursue other cases, as we have already found an answer.
- (2) To conclude that there is no counter-interpretation for the initial formula, we must pursue **all the possible cases**. If in **each and every** case we obtain a **contradictory tableau** (a tableau where the very same formula occurs in both columns), only then the procedure says:
YES, the initial formula is **valid**.

Exercises for the brave: the tableau algorithm indeed works

(These exercises are NOT exam material, though they are not too difficult.)

- (1) Give a precise argument showing that whatever formula A we take, if starting from A and ‘unfolding’ everything results in a non-contradictory tableau, then any interpretation I obtained by evaluating variables in the left column to **1** and in the right column to **0** is indeed always a counter-interpretation for the initial formula A .

Hint: Try to show the following two statements by induction on the formation of formulas: for every formula B occurring in the tableau,

- if B is in the left column, then B evaluates to **1** in I ,
- if B is in the right column, then B evaluates to **0** in I .

- (2) Give a precise argument showing that whatever formula A we take, if starting from A and ‘unfolding’ everything results in a contradictory tableau, then there is no counter-interpretation for A .

Hint: Try to show that if I is an interpretation where A evaluates to **0**, then we can always form a non-contradictory tableau by listing those subformulas of A that are **1** in I in the left column, and those that are **0** in I in the right column.

How to use tableaux for deciding satisfiability?

Question: Is formula A satisfiable?

In other words: Is there an interpretation where A evaluates to 1?

Tableaux are good with 'is there?' type of questions. This time we should begin with a tableau

A	
-----	--

Then we proceed as usual, by using the rules.

- If we end up with a non-contradictory tableau in one of the cases, then it gives us an interpretation where A is **1**. So in this case, YES, A is satisfiable.
- If all the cases finish with contradictory tableaux, then NO, A is not satisfiable.

How to use tableaux for deciding logical consequence?

Question: Is B a logical consequence of $A_1, A_2, \dots, A_n$?

Since tableaux are only good with 'is there?' type of questions, we should go after the opposite question: Is B **not** a logical consequence of $A_1, A_2, \dots, A_n$?

In other words: Is there an interpretation where each of $A_1, A_2, \dots, A_n$ evaluates to 1 but B evaluates to **0**? So this time we should begin with a tableau

A_1	B
A_2	
$\dots$	
A_n	

Then we proceed as usual, by using the rules.

- If we end up with a non-contradictory tableau in one of the cases, then it gives us an interpretation where $A_1, \dots, A_n$ are **1**, and B is **0**. So in this case, NO, B does not logically follow from $A_1, \dots, A_n$.
- If all the cases finish with contradictory tableaux, then YES, B does follow from $A_1, \dots, A_n$.

How to use tableaux for deciding logical equivalence?

Question: Are A and B logically equivalent?

Again, we should go after the opposite, 'is there?' type, question: Are A and B **not** logically equivalent?

In other words: Is there an interpretation where the truth-values of A and B are different? So this time we already have two cases at the start:

Case 1:

A	B
-----	-----

 or **Case 2:**

B	A
-----	-----

Then we proceed as usual, by using the rules.

- If we end up with a non-contradictory tableau in one of the cases, then it gives us an interpretation where the truth-values of A and B are different. So in this case, NO, A and B are not equivalent.
- If all the cases finish with contradictory tableaux, then YES, A and B are equivalent.

When Boolean logic is not too elegant

Consider the following argument:

Everyone in this room is feeling miserable today.

Therefore, Agi is feeling miserable today.

We can formalise this argument by choosing propositional variables

p : “James is feeling miserable today.”

q : “Sandeep is feeling miserable today.”

r : “Agi is feeling miserable today.”

z : “Zoe is feeling miserable today.”

...

Then we can translate, say, the assumption above as

$$p \wedge q \wedge r \wedge z \wedge \dots$$

and then the conclusion r logically follows.

This is rather cumbersome, but adequate. There are only finitely many people in the room, so $p \wedge q \wedge r \wedge z \wedge \dots$ (if we write it out fully) is a formula.

When Boolean logic is not enough

Consider the following argument:

- (1) *Every even number, except two, is not a prime.*
- (2) *Six is an even number and six is not two.*
- (3) *Hence, six is not a prime.*

We cannot use the 'trick' from the previous slide, as there are *infinitely* many numbers, but Boolean logic does allow *only finite* conjunctions.

- We can use **predicate logic** instead:

$$(1): \forall x (\text{EVEN}(x) \wedge \neg(x = 2) \rightarrow \neg\text{PRIME}(x))$$

$$(2): \text{EVEN}(6) \wedge \neg(6 = 2)$$

$$(3): \neg\text{PRIME}(6)$$

De Morgan's famous example

A horse is an animal.

Therefore, the head of a horse is a head of an animal.

A translation in predicate logic:

$$\forall x (\text{HORSE}(x) \rightarrow \text{ANIMAL}(x))$$

$$\forall y (\exists x (\text{HORSE}(x) \wedge \text{head}(x) = y) \rightarrow \exists x (\text{ANIMAL}(x) \wedge \text{head}(x) = y))$$

Predicate logic uses all the machinery of Boolean logic, plus:

- predicate symbols (such as EVEN, PRIME, HORSE),
- constants and function symbols (such as 6, head), and
- quantifiers over individual variables ($\forall y$ and $\exists x$).

Why to consider more kinds of logical models

- In *Boolean logic* all we can say about sentences that they are either true or false, in any given interpretation. Boolean logic does not allow for any further analysis of arguments, which may turn out to be inadequate from many points of view.
- *Predicate logic* provides us with a more detailed analysis of sentences and logical arguments. The price we pay for its increased expressiveness is its complex nature and undecidability:

There are NO effective, programmable methods for deciding validity, satisfiability, or logical consequence in predicate logic.

- Anything else?

Modal logics

- **Modal logics** are logical formalisms that are designed for handling different modes of truth ('**modalities**').
- There are many different kinds of modal logics: epistemic, temporal, dynamic, deontic, Each of them is tailored to describe and reason about different notions such as necessity, possibility, knowledge, belief, obligation, change in time, behaviour of programs, etc.
- Though all these features can be handled by predicate logic as well, modal logic gives a useful alternative to predicate logic.
 - For several application areas, it is more natural and intuitive to work with modal logic than with predicate logic.
 - In many cases, the logical procedures for modal logics are **effective** (that is, decidable, programmable), unlike for predicate logic.

Modes of truth

In natural language we often distinguish between various 'modes' of truth.

For example:

(1) '*Barack Obama is the president of the USA.*'

This is currently true, but it will not be **true some time in the future**.

(2) '*There are ten planets in our solar system.*'

While this is true(?), and maybe true forever in the future, is not **necessarily true** in the sense that it could have been a different number.

Modes of truth II

(3) ' $2 + 3 = 5$.'

This is true now and always in the future, and necessarily true in the sense that it follows from the laws of mathematics. But it does not enjoy all modes of truth: It may not be **known to be true** by e.g. small children; it may not be **believed to be true** by some people.

(4) '*It ought to be the case that we wear a seat-belt in a car.*'

This is only true depending on what **moral** codes we adopt.

(5) '*The value of the integer variable x is 19.*'

This is **true after some execution of a program** P (while might not be true for other executions of the same program).

Basic modal logic: alphabet

As before, we start describing a logic with defining its formal language (*syntax*).

The alphabet of basic modal logic consists of the following symbols:

- the symbols in the alphabet of Boolean logic (see slide 21)
- plus two new unary logical connectives:
 - (pronounced as 'box') and
 - ◇ (pronounced as 'diamond')

(Usually, □ and ◇ are called modal connectives or modalities.)

Syntax of basic modal logic: formulas

Well-formed **formulas** of basic modal logic (aka **modal formulas**) are built up inductively as follows:

- *Base case:* Any propositional variable is a modal formula.
- *Inductive step:* If A and B are modal formulas then so are

$$(\neg A) \quad (A \wedge B) \quad (A \vee B) \quad (A \rightarrow B) \quad (A \leftrightarrow B) \quad (\Box A) \quad (\Diamond A)$$

- *Closure:* Nothing else is a modal formula.

Examples:

- Any formula of Boolean logic is also a modal formula.
- $((\Box p) \wedge (\Box(\neg q))) \rightarrow r$

Notational conventions

- We omit the outermost parentheses of formulas.
- $\Box$, $\Diamond$ and $\neg$ bind more tightly than $\vee$, $\wedge$ and $\rightarrow$, so we omit the parentheses in occurrences of $(\Box A)$, $(\Diamond A)$ and $(\neg A)$.

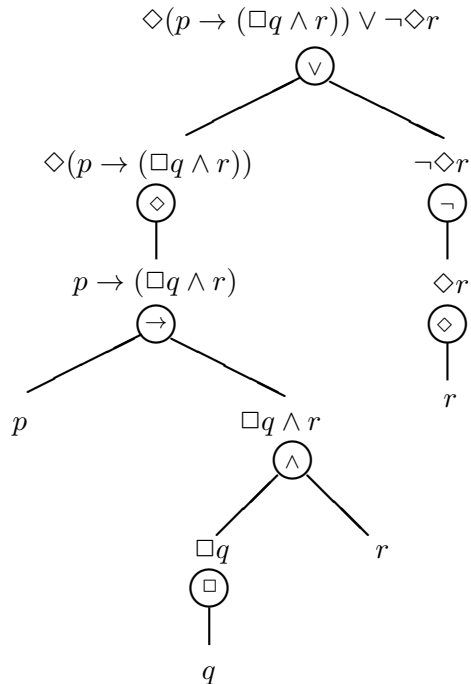
Examples:

we write $(\Diamond p \wedge \Box \neg q) \rightarrow r$ for $((\Diamond p) \wedge (\Box(\neg q))) \rightarrow r$

we write $\Box(p \wedge q) \vee (\Diamond r \wedge \neg \Box p)$ for $(\Box(p \wedge q)) \vee ((\Diamond r) \wedge (\neg(\Box p)))$

But always use brackets if in any doubt!

Parse tree of a formula



Any well-formed modal formula can be decomposed this way.

The formulas at the nodes of the parse tree are the **subformulas** of the initial formula.

Various readings of $\Box$ and $\Diamond$

$\Box A$:	$\Diamond A$:
I know A	A is compatible with my knowledge
A is necessary	A is possible
A will always be true	A will be true some time
A is obligatory	A is permitted
I believe A	A is compatible with my beliefs
A holds after any execution of my program P	A holds after some execution of my program P
...	...

Epistemic reading

$\Box A$ reads as '*I know A*' or '*A is known*' or '*agent Q knows A*'.

Modal formula	Reading
$\Box\Box p$	p is known to be known.
$\Box p \rightarrow p$	If p is known then p is true.
$\Box p \rightarrow \Diamond p$	If Q knows p then p is consistent with Q 's knowledge.
$\Box p \vee \Box \neg p$	Either Q knows p or Q knows not p .
$\Box(p \rightarrow q) \rightarrow (\Box p \rightarrow \Box q)$	? <i>logical omniscience</i>
$\Box p \rightarrow \Box\Box p$	? <i>positive introspection</i>
$\neg\Box p \rightarrow \Box\neg\Box p$	? <i>negative introspection</i>

Deontic reading

deontic = dealing with norms.

$\Box A$ reads as '*A is obligatory*' or '*A is ought to be true*',

$\Diamond A$ reads as '*A is permitted*'.

Modal formula	Reading
$(\Box p \wedge \Box q) \rightarrow \Box(p \wedge q)$	If both p and q are obligatory then so is ' p and q together'.
$\Box p \rightarrow \Diamond p$	If p is obligatory then p is permitted.
$p \rightarrow \Box p$	If p is true then p is obligatory.

Temporal reading

$\Box A$ reads as '*A will always be true in the future*'.

$\Diamond A$ reads as '*A will be true some time in the future*'.

Modal formula	Reading
$\Box p \rightarrow p$	If p will always be true then p is true now.
$\Box(p \rightarrow \Box p)$	Once p , always p .
$\Diamond \Box p$	Eventually permanently p .
$\Box(p \rightarrow \Diamond q)$	Whenever p , there is always a q afterwards.
$\Box \Diamond 1$	Time has no end.
$\Box 0$	Now it is the end of time.

Executions of a program

By an 'execution' of a program here we mean a *terminating* execution.

$\Box A$ reads as '*A holds after every execution of a program P*'.

$\Diamond A$ reads as '*A holds after some execution of a program P*'.

Modal formula	Reading
$(\Diamond p \wedge \Box q) \rightarrow \Diamond(p \wedge q)$	If there is an execution ending with p true, and every execution ends up with q true, then there is an execution with both p and q true.
$(\Diamond p \wedge \Diamond q) \rightarrow \Diamond(p \wedge q)$	If there is an execution with p true, and (possibly another) one with q true, then there is an execution with both p and q true.
$\Box(p \wedge \neg p)$	The program never terminates.

Multi-modal logics

Sometimes the setting we want to model needs *more than one* modality.

- **Multi-agent systems:** Different agents $Q_1, Q_2, \dots$ have different knowledge of the world. We need different modalities $\Box_1, \Box_2, \dots$ and $\Diamond_1, \Diamond_2, \dots$ to model this situation. Then we can have modal formulas like $\Box_1 \Box_2 p \rightarrow \Box_2 \Diamond_1 p$.
- **Reasoning about time:** We may want to speak about things in the past, not only in the future. Then we need a box modality $\Box_F$ for '*always in the future*' and another one $\Box_P$ for '*always in the past*', plus their diamond-variants $\Diamond_F$ and $\Diamond_P$.
- **Reasoning about programs:** In order to speak about the behaviour of different programs at the same time, we need different modalities $\Box_{P_1}, \Box_{P_2}, \dots$, and $\Diamond_{P_1}, \Diamond_{P_2}, \dots$, for each program $P_1, P_2, \dots$. This makes the logic suitable to speak about complex processes that are put together from simple parts.

Necessity

What if $\Box A$ reads as '*A is necessarily true*' ?

There are different kinds of necessities. For example, let p denote the statement

'The velocity of every material body is smaller than that of light.'

- Then the modal formula $\Box p$ expresses a true statement if $\Box$ is read as **physical necessity**: It follows from the laws of physics that p is true.
- However, if $\Box$ reads as **logical necessity**, then it is debatable whether $\Box p$ holds: It is logically possible that p is not true.
- Moreover, there are modal formulas, such as $\Box q \rightarrow \Box q$, that seem to be true *whatever* reasonable meaning we give to $\Box$.

~> **As before, we need a precise formulation of meaning.**

semantics: 'model circumstances'

‘Possible world’ semantics: the idea

Leibniz’ idea: necessity = truth in **all** possible worlds

possibility = truth in **at least one** possible world

So, we can imagine a system of possible *worlds* in which each world has some (possibly none) *alternatives*.

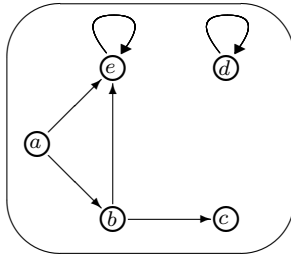
- Every world ‘lives under the laws’ of Boolean logic:
Each world has its own interpretation of the propositional variables, each of them are either true (**1**) or false (**0**) in it. The truth-values of compound formulas that are composed with the help of the Boolean logical connectives are determined by the usual truth-tables.
- Different worlds might have different interpretations of the same variables.
- A formula of the form $\Box A$ is regarded to be true in a world w , if A is true in **all** the worlds alternative to w .
- A formula of the form $\Diamond A$ is true in a world w , if A is true in **at least one** alternative of w .

Kripke frames

A Kripke frame is a pair $\mathfrak{F} = (W, R)$ where

- W is a non-empty set of worlds (or states, or points)
- R is a binary relation on W , the alternativeness (or accessibility) relation.

For example:



$$\mathfrak{F} = (W, R)$$

$$W = \{a, b, c, d, e\}$$

$$R = \{(a, b), (a, e), (b, c), (b, e), (d, d), (e, e)\}$$

If $(a, b) \in R$, then we say: ' a **sees** b ', or

' b **is accessible from** a ', or

' b **is an alternative for** a '.

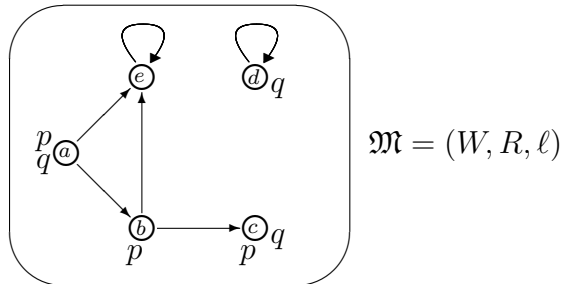
we also write: aRb

‘Model circumstances’ for modal logic: Kripke models

A Kripke model is a triple $\mathfrak{M} = (W, R, \ell)$ where

- (W, R) is a Kripke frame, and
- ℓ is a function labelling each world with the propositional variables that are supposed to be true in it. Such a labelling function ℓ is called a valuation.

For example:



- We say that model $\mathfrak{M} = (W, R, \ell)$ is based on the frame (W, R) .
- There can be many different models based on the same frame.

The meaning of modal formulas in Kripke models

Now we ‘connect’ the formal language and the ‘model circumstances’ of basic modal logic: We describe what modal formulas ‘mean’ in Kripke models.

For every modal formula A , Kripke model $\mathfrak{M}$ and world w in $\mathfrak{M}$, we will define the notions:

“ A is true in $\mathfrak{M}$ at world w ”

notation:

$$\boxed{\mathfrak{M} \models_w A}$$

“ A is false in $\mathfrak{M}$ at world w ”

$$\boxed{\mathfrak{M} \not\models_w A}$$

(There are other notations for this in the literature:

$(\mathfrak{M}, w) \models A$, or $\mathfrak{M}, w \Vdash A$, or $\|A\|_w^{\mathfrak{M}} = 1$, ...)

Evaluating modal formulas in Kripke models

Slogan: Every world w ‘lives under the laws’ of Boolean logic. So the ‘behaviour’ of Boolean logic connectives is determined by their usual truth-tables.

- If p is a propositional variable, then $\boxed{\mathfrak{M} \models_w p}$ if p is among the labels of w .
If p is not among the labels of w , then $\boxed{\mathfrak{M} \not\models_w p}$.
- $\boxed{\mathfrak{M} \models_w \neg A}$ if $\mathfrak{M} \not\models_w A$. Otherwise, $\boxed{\mathfrak{M} \not\models_w \neg A}$.
- $\boxed{\mathfrak{M} \models_w A \wedge B}$ if $\mathfrak{M} \models_w A$ and $\mathfrak{M} \models_w B$. Otherwise, $\boxed{\mathfrak{M} \not\models_w A \wedge B}$.
- The cases of $A \vee B$, $A \rightarrow B$, and $A \leftrightarrow B$ are similar, see their truth-tables:

A	B	$A \wedge B$	$A \vee B$	$A \rightarrow B$	$A \leftrightarrow B$
1	1	1	1	1	1
1	0	0	1	0	0
0	1	0	1	1	0
0	0	0	0	1	1

Evaluating modal formulas in Kripke models II

- A formula of the form $\Diamond A$ is regarded to be **true** at a world w , if A is true in **at least one** alternative of w :

$\mathfrak{M} \models_w \Diamond A$ if there is a world u in $\mathfrak{M}$ such that wRu and $\mathfrak{M} \models_u A$.

- Otherwise, $\mathfrak{M} \not\models_w \Diamond A$.

So, a formula of the form $\Diamond A$ is regarded to be **false** at a world w , if A is false at all the worlds alternative to w .

$\Diamond$ is a bit like $\exists$ of predicate logic

Special case: If world w is a 'dead-end' (a world with no alternatives), then $\Diamond A$ is **always false at** w (no matter what A is).

Evaluating modal formulas in Kripke models III

- A formula of the form $\Box A$ is regarded to be **true** at a world w , if A is true at **all** the worlds alternative to w :

$$\boxed{\mathfrak{M} \models_w \Box A} \quad \text{if} \quad \mathfrak{M} \models_u A \text{ for all worlds } u \text{ in } \mathfrak{M} \text{ with } wRu.$$

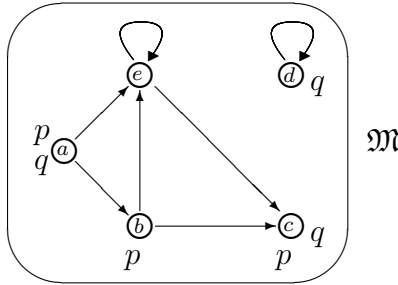
- Otherwise, $\boxed{\mathfrak{M} \not\models_w \Box A}$.

So, a formula of the form $\Box A$ is regarded to be **false** at a world w , if there is **at least one** world alternative to w where A is false.

$\Box$ is a bit like $\forall$ of predicate logic

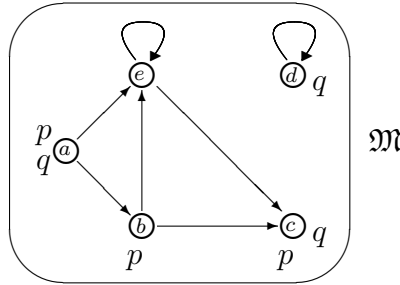
Special case: If world w is a 'dead-end' (a world with no alternatives), then $\Box A$ is **always true at** w (no matter what A is).

Exercise 5.1: model checking in Kripke models



- (1) $\mathfrak{M} \models_a p$: It is because p is among the labels of a .
- (2) $\mathfrak{M} \models_a \Box \neg q$: a has two alternatives, b and e . q is not among the labels of either of them, so $\mathfrak{M} \not\models_b q$ and $\mathfrak{M} \not\models_e q$, hence $\mathfrak{M} \models_b \neg q$ and $\mathfrak{M} \models_e \neg q$.
- (3) $\mathfrak{M} \not\models_e \Box q$: $e R e$ and $\mathfrak{M} \not\models_e q$.
- (4) $\mathfrak{M} \models_a \Diamond \Diamond (p \wedge q)$: $a R b$ and $b R c$, and $\mathfrak{M} \models_c p \wedge q$.
- (5) $\mathfrak{M} \not\models_a \Diamond \Diamond (p \wedge q) \wedge \Diamond q$: a has two alternatives, b and e , and q is false in both. So $\mathfrak{M} \not\models_a \Diamond q$.
- (6) $\mathfrak{M} \not\models_c \Diamond 1$: There is no world accessible from c , so all formulas of the form $\Diamond A$ are false in c .
- (7) $\mathfrak{M} \models_c \Box 0$: There is no world accessible from c , so all formulas of the form $\Box A$ are true in c (because we can't find a 'witness' to make them false).

Exercise 5.1: model checking in Kripke models (cont.)



- (8) $\mathfrak{M} \models_d \Diamond \Diamond q$: dRd and dRd , and $\mathfrak{M} \models_d q$.
- (9) $\mathfrak{M} \models_b \Diamond \Box \neg q$: bRc and c sees no other worlds, so all $\Box$ -formulas are true in c . In particular, $\mathfrak{M} \models_c \Box \neg q$.
- (10) $\mathfrak{M} \models_b \Diamond \Box q$: bRc and c sees no other worlds, so all $\Box$ -formulas are true in c . In particular, $\mathfrak{M} \models_c \Box q$.
- (11) $\mathfrak{M} \models_d \Diamond p \vee \Box q$: The only alternative of d is d , and $\mathfrak{M} \models_d q$, so $\mathfrak{M} \models_d \Box q$.
- (12) $\mathfrak{M} \not\models_e \Box(\Box q \vee \Box p)$: eRe , $\mathfrak{M} \not\models_e p$ and $\mathfrak{M} \not\models_e q$, so $\mathfrak{M} \not\models_e \Box p \vee \Box q$.

Truth and satisfiability

- A formula A is **true in a model** $\mathfrak{M}$ if

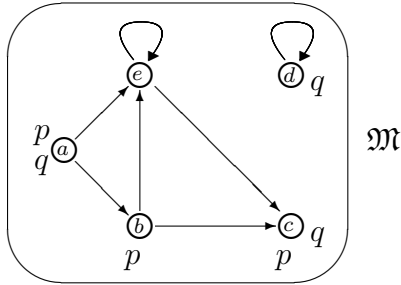
$$\mathfrak{M} \models_w A \text{ for all worlds } w \text{ in } \mathfrak{M}.$$

- A formula A is **satisfiable in a model** $\mathfrak{M}$ if

there is some world w in $\mathfrak{M}$ such that $\mathfrak{M} \models_w A$.

If A is true in $\mathfrak{M}$ then A is satisfiable in $\mathfrak{M}$.
--

Exercise 5.2: Truth and satisfiability



- (1) $\Box \neg p \wedge \Box \Box \neg p$: Satisfiable in $\mathfrak{M}$, as $\mathfrak{M} \models_c \Box \neg p \wedge \Box \Box \neg p$.
Not true in $\mathfrak{M}$, as $\mathfrak{M} \not\models_a \Box \neg p \wedge \Box \Box \neg p$.
- (2) $\Diamond(\neg p \vee \Diamond q)$: Satisfiable in $\mathfrak{M}$, as $\mathfrak{M} \models_a \Diamond(\neg p \vee \Diamond q)$.
Not true in $\mathfrak{M}$, as $\mathfrak{M} \not\models_c \Diamond(\neg p \vee \Diamond q)$.
- (3) $\Box p \rightarrow p$: True in $\mathfrak{M}$, as p is true in a, b and c , and $\Box p$ is false in d and e .
- (4) $\Box(p \leftrightarrow p)$: True in $\mathfrak{M}$.
Can you find a model where this formula is *not* true?

Validity and satisfiability in Kripke frames

- A formula A is **valid in a frame** $\mathfrak{F} = (W, R)$ if

A is true in *all* models $\mathfrak{M} = (W, R, \ell)$ based on $\mathfrak{F}$.

($\mathfrak{M} \models_w A$ for *all* worlds w in *all* models $\mathfrak{M}$ based on $\mathfrak{F}$.)

- A formula A is **satisfiable in a frame** $\mathfrak{F} = (W, R)$ if

A is satisfiable in *some* model $\mathfrak{M} = (W, R, \ell)$ based on $\mathfrak{F}$.

($\mathfrak{M} \models_w A$ for *some* world w in *some* model $\mathfrak{M}$ based on $\mathfrak{F}$.)

A is valid in $\mathfrak{F}$ iff $\neg A$ is not satisfiable in $\mathfrak{F}$
--

A is satisfiable in $\mathfrak{F}$ iff $\neg A$ is not valid in $\mathfrak{F}$
--

Valid modal formulas: examples

The following formulas are valid in all frames:

- (substitution instances of) valid formulas of Boolean logic
- $\Box p \leftrightarrow \neg \Diamond \neg p$
- $\Diamond p \leftrightarrow \neg \Box \neg p$
- $\Box 1$
- $\Box(p \rightarrow q) \rightarrow (\Box p \rightarrow \Box q)$ ($\leadsto$ this formula has a 'name': K)
- $\Diamond(p \rightarrow q) \rightarrow (\Box p \rightarrow \Diamond q)$
- $\Box(p \rightarrow q) \rightarrow (\Diamond p \rightarrow \Diamond q)$
- $\Box(p \wedge q) \leftrightarrow (\Box p \wedge \Box q)$
- $\Diamond(p \vee q) \leftrightarrow (\Diamond p \vee \Diamond q)$

Validity and effectiveness

Just like in the case of Boolean logic (see slide 33), we ask:

Is there an effective procedure (an algorithm) that decides whether a given modal formula is valid in **all** frames? By an *algorithm* we mean something meeting the following conditions:

- It must consist of exact step-by-step instructions.
- These instructions must not demand any brilliance or originality on the part of the person (or machine) following them. A diligent clerk (who does not like to think but is very good in following directions) or a computer (that does not think at all) should be able to execute the instructions.
- The procedure should be '**universal**': given **any** modal formula A , after a finite number of steps the procedure should produce either a "YES, A is valid in all frames" or "NO, A is not valid in all frames" answer.

In other words, we need a computer program that can take **any** modal formula as input, and outputs YES if the input formula is valid in all frames, and NO if it is not valid in all frames.

So is there such an algorithm?

Is there an effective procedure (algorithm, program) that decides whether a given formula is valid in **all** frames? Isn't it obvious that there is? NO!!

- The steps of 'formula-evaluation' on slides 136–139 can certainly be considered as 'algorithmic instructions': Given a (finite) Kripke model $\mathfrak{M}$, a world w in it, and a modal formula A , it is of course possible to check whether $\mathfrak{M} \models_w A$ or not. (It is not too difficult even to write a computer program for this task.)
- But how to decide whether a formula is valid in **all** frames? That is, whether $\mathfrak{M} \models_w A$ hold for *all* possible worlds in *all* possible models over *all* possible frames? There are frames having infinitely many worlds, and there are infinitely many different frames.
- Also, for each particular formula A , we might find some 'smart method' showing its validity in all frames. The problem is that we are looking for a 'universal method:' one which gives a correct answer for **all** formulas.

A validity-deciding algorithm for basic modal logic

We will extend the tableau procedure from Boolean logic to modal logic:

Given a modal formula A , we will try to show that it is **not valid in all frames**.

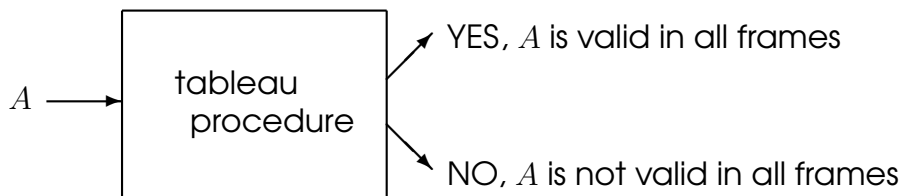
In other words, we will try find a countermodel for A :

- a frame $\mathfrak{F}$,
- a model $\mathfrak{M}$ based on $\mathfrak{F}$, and
- a world w in $\mathfrak{M}$

such that $\boxed{\mathfrak{M} \not\models_w A}$.

We will try to construct this countermodel via a finite number of simple but exactly given steps. These steps will not require any ‘thinking’, so even a computer can be programmed to follow them.

The tableau algorithm for basic modal logic



The procedure will be such that:

- It is **sound**: If it claims that something is a countermodel, then it indeed is.
So when the algorithm says NO, then the formula is indeed *not valid in all frames*.
- It is **complete**: It always finds a countermodel if it exists.
So if we do not succeed, then there is no countermodel for A , that is, A is *valid in all frames*. This is when the algorithm says YES.

Tableau procedure: step by step towards a countermodel

We try to construct the countermodel by building step-by-step a **tableau system**: a tree-like Kripke-frame of interconnected worlds, where each world has its own two-column tableau of formulas. For each world, we list:

- the formulas that we want to be **true at the world** in the **left** column, and
- the formulas that we want to be **false at the world** in the **right** column.

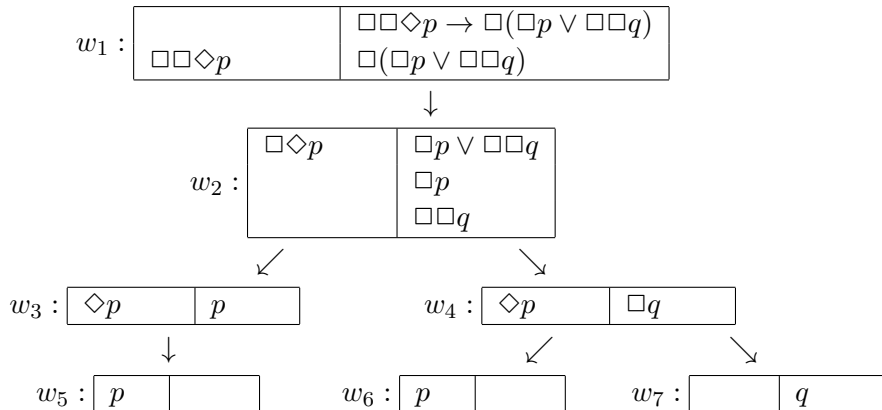


Tableau procedure: how to start

Question: Is the formula $\neg\Box\neg p \rightarrow \Box\neg\Box p$ valid in all frames?

- We are after a countermodel for $\neg\Box\neg p \rightarrow \Box\neg\Box p$, that is, a Kripke model and a world in it, where $\neg\Box\neg p \rightarrow \Box\neg\Box p$ is false.
- *At the beginning*, we don't know the 'world-structure' of such a possible countermodel, but if it exists, it must have at least one world where $\neg\Box\neg p \rightarrow \Box\neg\Box p$ is false.
- So we start the procedure with having just **one** world, say w_1 , where we want $\neg\Box\neg p \rightarrow \Box\neg\Box p$ to be false. So our initial tableau system looks like this:

$w_1 :$

	$\neg\Box\neg p \rightarrow \Box\neg\Box p$
--	---

Tableau procedure: what is next?

Starting from the initial tableau system consisting of just one world and its tableau, at each subsequent step:

- We choose **any** formula in one of the tableaux that is not a propositional variable and that hasn't been 'processed' before (until there is such).
- We find its 'outermost' logical connective.
- Then apply the corresponding rule. A rule:
 - either puts some subformulas into the columns of some of the existing tableaux (not necessarily at the same world as the chosen formula),
 - or sometimes
 - creates a new world with a new two-column tableau,
 - connects the new world with an already existing world with an arrow, and
 - puts some subformulas into some column of the new tableau.
- The order of choosing formulas for 'unfolding' does not matter.

Tableau procedure: the end

Just like in the tableau procedure for Boolean logic, there will be rules that result in **different cases**, to be pursued in parallel. This time each case may consist of not just one tableau but a whole tableau system.

However, overall there can be **two reasons** for the procedure to stop:

- (1) In **one of the cases**, there are no 'unfoldable' formulas left in any of the tableaux of the system. In such a case we can turn the tableau system to a countermodel for the initial formula (see slide 161 below).

The procedure says: NO, the initial formula is **not valid in all frames**.

- (2) In **every case**, we obtain a *contradictory tableau at some of the worlds* of the system: a tableau where the very same formula occurs in *both* its left (true) and right (false) column. We would like this to mean that there is no countermodel for the initial formula. (?? see later why)

The procedure says: YES, the initial formula is **valid in all frames**.

Tableau procedure: the rules

There are altogether 14 rules, 2 for each of the 7 logical connectives

$\neg, \wedge, \vee, \rightarrow, \leftrightarrow, \Diamond$, and $\Box$.

- The rules for the Boolean connectives $\neg, \wedge, \vee, \rightarrow$, and $\leftrightarrow$ are the same as before (see slides 47–48). These rules always work ‘world-wise:’

When we apply these rules to a formula in a tableau at a world,

- the resulting formula(s) will be put into the *same* tableau, and
 - there are *no changes* to tableaux at *other* worlds.
- There are 4 new rules:
 - the $\Diamond_{true}$ rule,
 - the $\Box_{false}$ rule,
 - the $\Diamond_{false}$ rule,
 - the $\Box_{true}$ rule.

Let’s see them one by one.

Tableau procedure: the $\Diamond_{true}$ rule

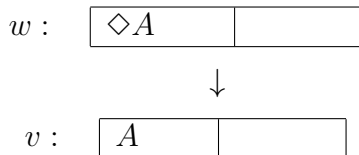
Recall from slide 139:

$\Diamond A$ is true in world w only if

there is some world v accessible from w such that A is true in v .

So, whenever we choose a formula of the form $\Diamond A$ in the 'true' column, then:

- we create a **new world** with a **new tableau**,
- we make the new world, say v , **accessible** from the world where we chose $\Diamond A$, and
- put A into the 'true' column of the new tableau.



The $\Diamond_{true}$ rule: an example

Recall that we want to interpret our failure in finding a countermodel as a **proof** for not having such at all. So we are always after a 'generic' or 'most general' situation.

In particular, if we have several **different** $\Diamond$ -formulas in the 'true' column, then **each** such $\Diamond$ -formula 'asks for' a **different** new world 'witnessing' its truth.

For example, three subsequent applications of the $\Diamond_{true}$ rule give us the tableau system:

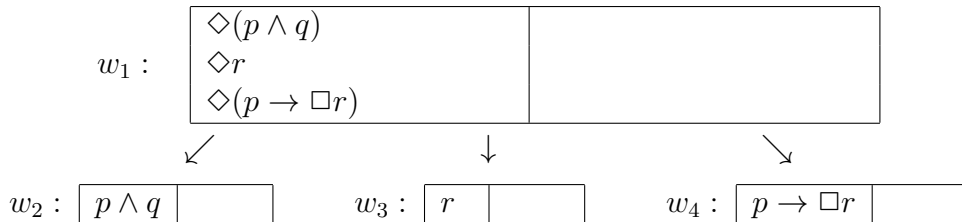


Tableau procedure: the $\Box_{false}$ rule

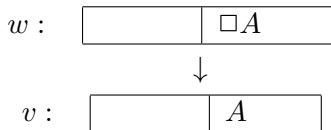
This is 'mirroring' the $\Diamond_{true}$ rule. Recall from slide 138:

$\Box A$ is false in world w only if

there is some world v accessible from w such that A is false in v .

So, whenever we choose a formula of the form $\Box A$ in the 'false' column, then:

- we create a **new world** with a **new tableau**,
- we make the new world, say v , **accessible** from the world where we chose $\Box A$, and
- put A into the 'false' column of the new tableau.



If we have several **different** $\Box$ -formulas in the 'false' column, then **each** such $\Box$ -formula 'asks for' a **different** new world 'witnessing' its falsity.

Tableau procedure: the $\Box_{true}$ rule

Recall from slide 138:

$\Box A$ is true in world w only if A is true in v , for every world v that is accessible from w .
In particular, if there are *no* worlds accessible from w .

Slogan: We only create new worlds when we are 'forced to'.

So, whenever we choose a formula of the form $\Box A$ in the 'true' column of a world w :

- If at the moment no worlds are accessible from w , then we do nothing.
(But we must check back later, because some other rules might create new worlds that become accessible from w .)
- If there are worlds accessible from w , then at each and every such world, we put A into the 'true' column.
(Again, we must check back later, because some other rules might create more worlds that become accessible from w .)

So the $\Box_{true}$ rule is a '**backtracking**' rule: Any time we have a $\Box$ -formula in the 'true' column of one of the tableaux of our system, we have to re-check it time and time again.

Tableau procedure: the $\Diamond_{false}$ rule

This is ‘mirroring’ the $\Box_{true}$ rule. Recall from slide 139:

$\Diamond A$ is false in world w only if A is false in v , for every world v that is accessible from w .
In particular, if there are *no* worlds accessible from w .

So, whenever we choose a formula of the form $\Diamond A$ in the ‘false’ column of a world w :

- If at the moment no worlds are accessible from w , then we **do nothing**.
(But we must check back later, because some other rules might create new worlds that become accessible from w .)
- If there are worlds accessible from w , then at **each and every** such world, we put A into the ‘false’ column.
(Again, we must check back later, because some other rules might create more worlds that become accessible from w .)

So the $\Diamond_{false}$ rule is a ‘**backtracking**’ rule: Any time we have a $\Diamond$ -formula in the ‘false’ column of one of the tableaux of our system, we have to re-check it time and time again.

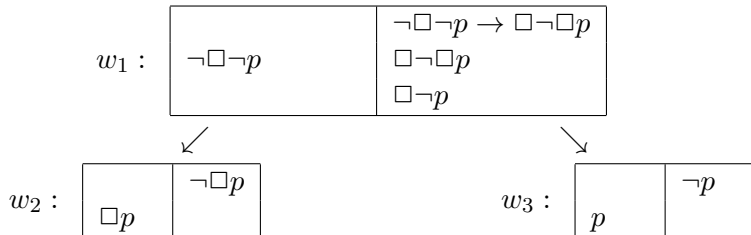
Tableau procedure: example continued

We continue the initial tableau system of slide 151:

$$w_1 : \begin{array}{|c|c|} \hline & \neg \Box \neg p \rightarrow \Box \neg \Box p \\ \hline \end{array}$$

We apply:

- First, the $\rightarrow_{false}$ rule to $\neg \Box \neg p \rightarrow \Box \neg \Box p$ at w_1 .
- Then, the $\neg_{true}$ rule to $\neg \Box \neg p$ at w_1 .
- Then, the $\Box_{false}$ rule twice, to $\Box \neg \Box p$ at w_1 (creates w_2), and to $\Box \neg p$ at w_1 (creates w_3).
- Then, the $\neg_{false}$ rule twice, to $\neg \Box p$ at w_2 , and to $\neg p$ at w_3 .



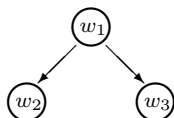
- There is no world accessible from w_2 , so the $\Box_{true}$ rule applied to $\Box p$ at w_2 does not give us anything else.

Tableau procedure: turning the result to a countermodel

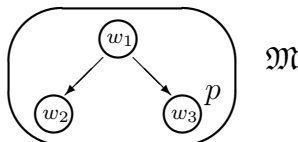
There is nothing left to 'unfold,' the procedure stops: the initial formula is *not valid in all frames*.

We can turn the resulting non-contradictory tableau system to a countermodel $\mathfrak{M}$ for $\neg\Box\neg p \rightarrow \Box\neg\Box p$:

- The frame of $\mathfrak{M}$ is the 'frame-structure' of the tableau system:



- Every world is labelled by those propositional variables that occur in the 'true' column of the tableau at the world in question:



We can always test whether we worked correctly: We should check whether the initial formula is indeed false at the initial world of our model.

In this case, whether $\neg\Box\neg p \rightarrow \Box\neg\Box p$ is false in $\mathfrak{M}$ at world w_1 . **(check it!)**

Exercise for the brave

(This exercise is NOT part of the exam, though it is not too difficult.)

Give a precise argument showing that whatever modal formula A we take, if starting from A and ‘unfolding’ everything results in a non-contradictory tableau system, then the model $\mathfrak{M}$ obtained from it as above (see slide 161) is indeed always a countermodel for the initial formula A .

Hint: Try to show the following two statements by induction on the formation of formulas: for every world x in $\mathfrak{M}$ and every formula B occurring in the tableau at x ,

- if B is in the left column, then $\mathfrak{M} \models_x B$,
- if B is in the right column, then $\mathfrak{M} \not\models_x B$.

Tableau procedure: another example

Question: Is the formula $\Box(p \rightarrow q) \rightarrow (\Box p \rightarrow \Box q)$ valid in all frames?

We know it is (this is one of the formulas listed on slide 145).

But let us see what happens, if we try to construct a countermodel.

We apply:

- The $\rightarrow_{false}$ rule twice, to $\Box(p \rightarrow q) \rightarrow (\Box p \rightarrow \Box q)$ and then to $\Box p \rightarrow \Box q$ at w_1 .
- Then, the $\Box_{false}$ rule to $\Box q$ at w_1 (creates w_2).
- Then the $\Box_{true}$ rule twice, to $\Box(p \rightarrow q)$ at w_1 , and then to $\Box p$ at w_1 .

$w_1 :$	<table><tr><td>$\Box(p \rightarrow q)$ $\Box p$</td><td>$\Box(p \rightarrow q) \rightarrow (\Box p \rightarrow \Box q)$ $\Box p \rightarrow \Box q$ $\Box q$</td></tr></table>	$\Box(p \rightarrow q)$ $\Box p$	$\Box(p \rightarrow q) \rightarrow (\Box p \rightarrow \Box q)$ $\Box p \rightarrow \Box q$ $\Box q$
$\Box(p \rightarrow q)$ $\Box p$	$\Box(p \rightarrow q) \rightarrow (\Box p \rightarrow \Box q)$ $\Box p \rightarrow \Box q$ $\Box q$		
	$\downarrow$		
$w_2 :$	<table><tr><td>$p \rightarrow q$ p</td><td>q</td></tr></table>	$p \rightarrow q$ p	q
$p \rightarrow q$ p	q		

Example continued: the $\rightarrow_{true}$ rule

Recall the $\rightarrow_{true}$ rule from slide 48. There are two cases:

Case 1.

$A \rightarrow B^\bullet$	A
---------------------------	-----

Case 2.

$A \rightarrow B^\bullet$ B	
----------------------------------	--

Our example:

Case 1.

$w_1 :$

$\Box(p \rightarrow q)$	$\Box(p \rightarrow q) \rightarrow (\Box p \rightarrow \Box q)$
$\Box p$	$\Box p \rightarrow \Box q$
	$\Box q$

↓

$w_2 :$

$p \rightarrow q^\bullet$	q
p	p

Case 2.

$w_1 :$

$\Box(p \rightarrow q)$	$\Box(p \rightarrow q) \rightarrow (\Box p \rightarrow \Box q)$
$\Box p$	$\Box p \rightarrow \Box q$
	$\Box q$

↓

$w_2 :$

$p \rightarrow q^\bullet$	q
p	
q	

Both cases are contradictory, the procedure **stops**.

Tableau procedure: detecting validity

What conclusion can we draw?

In every case, at world w_2 we have a *contradiction*: a propositional variable that is ought to be true and false at the same time, which is not possible.

<i>Conclusion:</i> $\Box(p \rightarrow q) \rightarrow (\Box p \rightarrow \Box q)$ is valid in all frames .
--

Why?? Sloppy answer:

We did our best, but did not succeed to construct a countermodel.
So there is no such.

In this particular case this argument might be convincing. But can we conclude that the tableau procedure **always** works?

YES. But a precise general proof is out of our scope.

Exercise for the even more ambitious

(This is a bit harder, and NOT part of the exam either.)

Show that if there is a countermodel for a formula A , then the tableau procedure will find it. (In fact, if there is a countermodel for a formula A , then it is possible to construct a non-contradictory tableau system starting from A that contains not more than 2^n worlds, where n is the number of subformulas of A .)

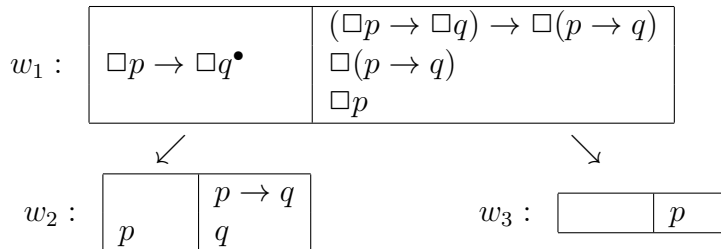
Ask for hints if interested.

Tableau procedure: yet another example

Question: Is the formula $(\Box p \rightarrow \Box q) \rightarrow \Box(p \rightarrow q)$ valid in all frames?

We apply:

- The $\rightarrow_{false}$ rule to $(\Box p \rightarrow \Box q) \rightarrow \Box(p \rightarrow q)$ at w_1 .
- Then, the $\Box_{false}$ rule to $\Box(p \rightarrow q)$ at w_1 (creates w_2).
- Then, the $\rightarrow_{false}$ to $p \rightarrow q$ at w_2 .
- Then, the $\rightarrow_{true}$ to $\Box p \rightarrow \Box q$ at w_1 . There are two cases.
- **Case 1.** Then, the $\Box_{false}$ rule to $\Box p$ at w_1 (creates w_3).

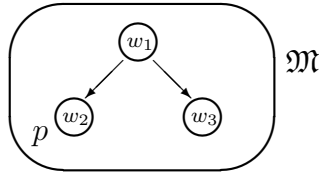


Example continued: the countermodel

There are no 'unfoldable' formulas left, so the procedure stops:

$(\Box p \rightarrow \Box q) \rightarrow \Box(p \rightarrow q)$ is *not valid in all frames*.

We can turn the resulting non-contradictory tableau system to a countermodel $\mathfrak{M}$ for $(\Box p \rightarrow \Box q) \rightarrow \Box(p \rightarrow q)$:



(As we already have a countermodel, we do not have to check Case 2 of the application of the $\rightarrow_{true}$ rule to $\Box p \rightarrow \Box q$ at w_1 .)

Test: Show that indeed $\mathfrak{M} \not\models_{w_1} (\Box p \rightarrow \Box q) \rightarrow \Box(p \rightarrow q)$.